



山东大学
SHANDONG UNIVERSITY

WaterOS 内核设计与优化实现文档

参赛队名: OuterSystems

队伍成员: 宋浩宇、李佳灿、孙馨宇

指导老师: 颜廷坤、潘润宇

目 录

第 1 章	我们为什么开始做这轮优化	1
1.1	最终纳入的优化范围	1
第 2 章	基线、定位方法和取舍标准	2
2.1	从现象到根因	2
第 3 章	我们保留下来的核心优化路径	3
3.1	先把重复的路径和文件访问收起来	3
3.2	再处理 exec、ELF 和 TLB 的前台成本	3
3.3	减少任务、同步和地址空间查询的固定成本	3
3.4	把物理页清零移出前台分配路径	4
第 4 章	最终结果和实验分析	5
4.1	端到端结果	5
4.2	结果应该怎样解读	5
4.3	没有纳入最终组合的实验	5
第 5 章	AI 使用说明与复现步骤	6
5.1	AI 在开发过程中的位置	6
5.2	复现环境	6
5.3	复现流程	6
第 6 章	总结	7

第 1 章 我们为什么开始做这轮优化

这份文档记录的是我们为了跑通并加速比赛测例，对 WaterOS 内核做的一轮实际优化。我们没有把所有实验分支都写进来。仓库里的 `perf/*` 分支记录了大量候选方案，其中有些只改善了一个局部符号，有些在短测试中有效但完整 BuildStorm 反而变慢，最后没有进入 main。正文只讲已经合入 main、并且在最终工作流中保留下来的优化；被回退的方案只在最后作简短说明。

我们最初面对的不是一个单独的慢函数。测例要经过用户程序、系统调用、任务、虚拟内存、VFS、文件系统和块设备，多处小开销叠在一起后，才形成了完整编译时间。我们的做法因此也不是先挑一个看起来漂亮的数据结构，而是先固定 QEMU 和镜像，建立可以重复的端到端基线，再用低开销计数和短路径采样缩小范围。每完成一条优化，我们都回到完整 workload 上做 A/B；局部指标好看但完整结果没有收益的改动，我们就回退。

本次工作的主要结果包括：修复影响测例得分的功能和稳定性问题；减少 syscall 到 VFS/FS 读路径中的重复解析和重复查询；降低 exec、ELF 装载和 mmap/munmap 的页表维护成本；减少调度、进程生命周期和同步路径上的全局开销；最后把物理页清零从前台分配路径移到 idle CPU，并维护一个已清零页面池。后文按这条主线叙述，而不是按提交日期逐条罗列。

1.1 最终纳入的优化范围

表 1-1 正文覆盖的最终优化主线

方向	我们最终保留的工作
测量与基线	固定 QEMU 9.2.1、镜像和 -snapshot，建立完整 BuildStorm 与 pc-hot 两级实验方法。
VFS 与读路径	复用路径解析结果，减少 dentry/metadata 重复查询，建立稳定的只读访问通道，降低 read staging 分配。
exec 与内存映射	复用 exec 文件句柄，启用 lazy ELF，按实际 PTE 变化汇总 mmap/munmap 的 TLB 同步。
任务与同步	保留有效的地址空间/当前任务快路径、单页 COW 刷新及 futex registry 分片等改动。
物理页分配	由 idle CPU 预先清零页面并放入空页池，前台分配优先命中已清零页面。

第 2 章 基线、定位方法和取舍标准

我们把“能不能跑完”和“跑得快不快”分开记录。前者使用功能测例和完整阶段回归确认,后者使用同一宿主、同一 QEMU、同一镜像和同一输入做交错 A/B。完整 BuildStorm 的计时从 workload 启动后的统一标记开始,到内核报告 BUILDSTORM_COMPILE mode=multi ok=true elapsed_s 为止;短 pc-hot 只用来筛选热点,不作为最终验收结果。

正式实验使用 QEMU 9.2.1、RISC-V QEMU virt、8 个 vCPU、16 GiB 内存和带 -snapshot 的发布镜像。每轮记录提交号、内核和镜像哈希、QEMU 参数、开始和结束时间以及完整日志路径。涉及架构无关代码时,我们至少执行 RISC-V 和 LoongArch 的静态检查;涉及文件系统、MM、task 生命周期的改动,还要执行对应的 final kernel 和功能回归。

我们采用一个很实际的取舍标准:完整 workload 有稳定收益且没有 panic、SIGSEGV、OOM、文件系统错误或测例缺项,才允许合入 main。低于宿主噪声的变化记为“无可确认收益”。例如,某些路径规范化和时间换算实验确实降低了局部指令数,但完整轮没有改善,所以没有放进最终方案。

2.1 从现象到根因

我们的定位过程大致固定为四步。第一步是在完整日志中确认首个异常或耗时阶段;第二步用计时和低开销计数确认调用次数、锁等待或页表操作是否真的集中;第三步做一个只改变单一因素的 A/B;第四步回到完整轮,检查局部收益是否能传递到端到端结果。这样做有点慢,却能避免把“热点”误当成“根因”。

在这条过程中,我们逐渐确认了几类重复成本:同一个路径在一次系统调用链中被多次解析,同一个文件的 metadata 被不同层重复读取;ELF 和 mmap 路径在没有实际 PTE 变化时仍进行同步;前台页分配承担了整页清零;而一些轻量查询仍然绕过同一份全局 registry 锁。后续章节按这些根因展开。

第 3 章 我们保留下来的核心优化路径

3.1 先把重复的路径和文件访问收起来

最早一批有效改动集中在 VFS 到文件系统的读路径。我们在采样中看到，BuildStorm 会反复打开、查询和读取相同的稳定文件；原有调用链每次都重新走路径分量、mount 路由和 metadata 查询。单次调用并不慢，但它们位于高频 syscall 链上，累计后占据了明显时间。

我们的处理方式不是在 syscall 层复制一份缓存，而是让一次 lookup 产生可复用的稳定结果：节点身份、metadata 和 mount identity 随解析结果向下传递，openat、stat 家族和后续 read 可以消费同一份结果。正 dentry 使用受控的淘汰策略，负 dentry 和页缓存则按真实访问模式限制容量。对根文件系统的稳定只读操作，我们补上了不依赖全局可变 FS 锁的访问契约，再把普通 read 的 staging buffer 重用起来。

这条链的根因是重复工作，不是简单的“缓存越大越好”。我们做过增大缓存、改 key 形式和路径快速分支的 A/B；其中有些方案减少了 memcmp，却增加了总指令或带来回归，因此没有合入。最终保留下来的方案共同减少了解析、分配和重复拷贝，并保持了 close、rename、truncate、fsync 等生命周期边界。

3.2 再处理 exec、ELF 和 TLB 的前台成本

当文件读路径收敛后，pc-hot 把我们带到 exec 和地址空间建立路径。原实现中，exec 前缀、ELF 头和 PT_LOAD 信息会通过不同层重复取得；lazy mmap 即使没有实际修改 PTE，也可能触发不必要的 TLB 同步。

我们先让 exec 在稳定文件句柄上复用已经读取的数据，再分别在 RISC-V 和 LoongArch 上处理 lazy ELF 的架构约束。随后，mmap 和 munmap 不再按调用次数盲目 fence，而是汇总真实发生的 PTE 变化，只在需要时同步。COW 缺页也从整段刷新缩小到单页失效。这些改动的共同点是：把“可能需要同步”改成“确实发生变化才同步”，同时保留缺页、权限和跨架构返回路径的正确性。

3.3 减少任务、同步和地址空间查询的固定成本

BuildStorm 会创建和退出大量短生命周期任务。我们在这里遇到过几个看起来合理但最终无效的方向：减少 snapshot 构造并没有绕开 scheduler registry 的全局锁，增加 per-parent child index 反而让完整轮变慢。最终保留的改动集中在真正的固定成本上，包括当前地址空间按 CPU 缓存、单页 COW 刷新，以及在确认不会破坏 lost-wake 语义后对 futex registry 做分片。

这一阶段我们格外注意锁的所有权。查询路径不在锁内做用户拷贝、调度或文件系统操作；分片只改变 registry 的竞争范围，不改变 futex key 的匹配和唤醒顺序。对没有完整端到端收益的 scheduler 轻量查询，我们没有因为代码更短就保留。

3.4 把物理页清零移出前台分配路径

最后一项收益明显的优化来自物理页分配。用户匿名页、零填充映射和页表页必须得到清零页面，原实现由发起分配的 CPU 同步完成清零。我们在单轮实验中看到，完整 Build-Storm 从约 530 s 降到约 502 s，说明这段工作已经成为前台高频成本。

我们增加了一个全局固定容量的 LIFO 空页池。idle CPU 在进入 WFI 前尝试从普通 frame allocator 批量取得 raw frame，在池锁外完成整页清零，再把页面发布到池中；前台分配先 pop 已清零页面，池不足时才回退到同步清零。当前正式设计使用 1024 页容量、256 页低水位、idle 补充批量 32 页和同步 miss 补充批量 16 页。池是可回收缓存，不改变 raw frame 的“内容未定义”契约；已经被写入后释放的页面回普通 allocator，不回流到空页池。

并发约束比池本身更重要。一个 PPN 同一时刻只能属于普通 allocator、清零中的 producer、空页池或调用者之一；清零期间 producer 独占唯一引用；不同时持有 allocator 锁和 pool 锁；池维护拿不到 allocator 锁时直接跳过，不自旋。这样即使 idle CPU 没有足够空闲时间，系统也只是退回原来的同步路径，不会把内存不足变成加载失败。

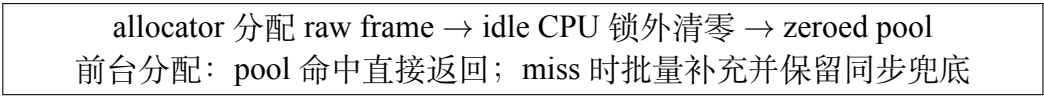


图 3-1 预清零物理页的所有权转移路径

第 4 章 最终结果和实验分析

4.1 端到端结果

我们最终采用完整 BuildStorm 耗时作为主指标，因为短 pc-hot 只能告诉我们某个符号变热或变冷，不能证明比赛测例真的变快。已确认的空页池 A/B 结果如下：

表 4-1 预清零页面池的代表性 A/B 结果

配置	完整编译时间	加速比	耗时降低
优化前：前台同步清零	530 s	1.000×	–
优化后：idle 预清零页面池	502 s	1.056×	5.28%

加速比按

$$S = \frac{T_{\text{before}}}{T_{\text{after}}}$$

计算。这里的 530 s 和 502 s 是相同 workload 条件下的代表性完整轮结果；如果后续补充多轮交错 A/B，我们会以中位数替换单轮值，并保留原始日志路径。

4.2 结果应该怎样解读

这 28 秒不是由某一个短函数直接贡献的。页面清零被放到 idle CPU 后，前台 fault、页表建立和匿名映射路径少了一段同步工作；同时，批量从 allocator 取得 raw frame 也减少了全局分配器锁的获取次数。空页池优化因此和前面的 exec、mmap、page cache 改动存在路径上的叠加，但我们不把各项百分比简单相加，最终只报告完整组合和可复核的 A/B。

编译时间本身不是本轮所有优化的独立目标。我们主要优化内核运行时，因此构建时间数据只在同一宿主、同一工具链和同一配置下比较；不能把宿主编译缓存、QEMU 启动时间或不同镜像状态混入结论。功能回归、双架构 check 和 snapshot 运行是性能数据的前置条件，任何出现 panic、文件系统错误或测例缺项的候选都不进入最终组合。

4.3 没有纳入最终组合的实验

我们保留了失败实验的记录，但没有把它们写成成果。例如，process child index、u128 时间换算、路径规范化快速分支和 TLSF 懒统计都出现过局部指标改善；完整轮却没有稳定收益，甚至出现退化，所以最终回退。这个取舍很重要：我们愿意放弃一个局部漂亮的数字，换取完整 workload 上可重复的结果。

第 5 章 AI 使用说明与复现步骤

5.1 AI 在开发过程中的位置

我们使用 AI 作为代码阅读、调用链整理、实验方案讨论和文档整理的辅助工具，而不是把 AI 输出直接当成实现结论。AI 可以根据日志和源码提出可能的热点，但是否修改由我们决定；每个候选都要经过 diff 审查、编译检查、功能回归和完整 A/B。对 AI 建议的“局部更快”，如果完整轮没有收益，我们就记录并回退。

在空页池优化中，AI 主要帮助我们整理 allocator、idle loop、page fault 和 frame ownership 之间的调用关系，并检查池为空、OOM drain、锁顺序和页面生命周期等边界。容量、水位、批量大小以及是否合入，仍由我们根据实际测量和代码审查决定。

5.2 复现环境

复现时应使用仓库中的对应 main 提交，并固定以下条件：QEMU 9.2.1、RISC-V virt、8 vCPU、16 GiB 内存、发布内核、同一份赛题镜像和 -snapshot。两次 A/B 只允许改变待测提交，不能更换镜像或 QEMU 参数。

5.3 复现流程

下面的命令示意完整流程，实际 runner 路径以仓库当前任务记录为准：

1. 分别检出优化前和优化后的提交，记录 `git rev-parse HEAD`。
2. 在 `os/` 中执行 `make rv_check`；跨架构改动还执行 `make la_check`。
3. 构建对应 final kernel，并记录内核 SHA-256。
4. 使用 `os/scripts/perf/buildstorm_runner.py` 启动完整 BuildStorm，指定相同的镜像、`-arch rv`、`-timeout` 和输出目录；运行参数必须保留 `-snapshot`。
5. 从日志中提取 `BUILDSTORM_COMPILE mode=multi ok=true elapsed_s`，重复运行并交错排列 A/B。
6. 对比 T_{before} 和 T_{after} ，按式 $S = T_{\text{before}}/T_{\text{after}}$ 计算加速比，并保存完整日志。

提交和任务记录是复现开发过程的入口：`docs/tasks/perf/` 保存各阶段的方案、验收和回退原因，`docs/agents/tasks/syscall-chain-opt/` 保存 syscall 链优化的逐项简报。最终文档只抽取进入 main 的路径，审核者可以沿 commit 和这些记录继续核对源码与原始实验。

第 6 章 总结

回头看这轮优化，我们并不是从一个“万能优化”开始的。先是让测例稳定跑完，再建立固定的完整轮 A/B；随后沿着实际热点从 `syscall/VFS` 读路径一路追到 `exec`、页表、任务查询和物理页分配。很多候选方案只在局部统计中变好，最终没有进入 `main`。真正留下来的改动都有同一个条件：能解释清楚根因，能保持正确性，并且在完整 `workload` 上重复得到收益。

预清零页面池是这条路径的最后一个例子。我们没有改变页面分配对调用者的语义，只把必须做的清零工作交给 `idle CPU`，并在池为空时保留同步兜底。代表性完整轮从约 530s 降到约 502s，说明内核底层的固定成本仍然会传递到用户态编译时间。

本文没有替代仓库中的任务简报，也没有把所有实验记录重复一遍。它只提供最终实现的因果链、关键设计和可复核入口；详细 `commit`、命令和回退证据仍以 `main` 历史以及 `docs/tasks/perf/`、`docs/agents/tasks/syscall-chain-opt/` 中的记录为准。